```
        }

        /// <summary>
        /// Vector negation.
        /// </summary>
        public static Vec operator -(Vec vec)
        {
            return new Vec(-vec.X, -vec.Y);
        }
    }
```

Note the shape of the overloaded operator methods. They are declared public static and take the expected number of arguments.

You can overload a number of unary and binary operators:

```
Unary: +, -, !, ~, ++, --, true, false
Binary: +, -, , /, %, &, |, ^, <<, >>, ==, !=, >, <, >=, <=
```

The restrictions you might expect apply for certain operators, e.g.,

Overloading ++ on a type T requires that you return a `T`. If you overload `!=` you must also overload ==.

See the MSDN documentation for more information on these restrictions. In addition to operators, you can overload array notation to provide list-like indexing into your classes. These are called indexers.

```
struct Pair<T, U>
{
    public T Fst { get; private set; }
    public U Snd { get; private set; }
    public Pair (T t, U u) : this()
    {
        Fst = t;
        Snd = u;
    }
}

class SimpleMap<T, U>
{
    List<Pair<T, U>> list;
    public SimpleMap()
    {
```